

Question 1

Soit M la taille de l'espace des mots de passe.

Technique	Pré-calcul (temps)	Pré-calcul (espace)	Attaque (temps)	Attaque (espace)
Force brute	–	–	$O(26^M)$ hachages($\frac{26^M}{2}$ en moyenne)	$O(M)$
Base pré-calculée	$O(26^M)$ hachages, plus un tri en $O(26^M \cdot M)$	$O(26^M \cdot (M + n))$	$O(M)$ ou $O(1)$ avec une table de hachages.	$O(26^M \cdot (M + n))$

Soit M la longueur des mots de passe; l'espace contient $N = 26^M$ mots de passe.

Force brute : on hache chaque mots des passes jusqu'à trouver le bon ($\frac{26^M}{2}$ essais en moyenne).

Base pré-calculée : on stocke une fois tous les couples $(h(x), x)$, puis l'attaque est une simple recherche. Le coût est amorti, mais la mémoire est en $O(26^M)$.

Valeurs réalistes : un GPU fait environ 2^{33} hashes/s et $26^M \approx 2^{4,7M}$, donc la force brute reste faisable jusqu'à $M \approx 10$ – 11 (environ 2^{50} essais, environ 36h). Pour la base, avec environ 32 octets par entrée, 1 To contient environ 2^{35} entrées on est donc limité par le stockage vers $M \approx 7$ – 8 .

Question 2

On cherche systématiquement un mot de passe *final* $pass_{x,L}$ parmi les N chaînes stockées, afin de récupérer le $pass_{x,0}$ correspondant. La structure doit donc être indexée sur $pass_{x,L}$ (la clé) et renvoyer $pass_{x,0}$ (la valeur).

On utilise une **table de hachage** (dictionnaire) $pass_{x,L} \rightarrow pass_{x,0}$, qui permet une recherche en $O(1)$ en moyenne.

Sans une telle structure, il faudrait parcourir linéairement les N couples à chaque test de point final ($O(N)$ par recherche), ce qui, répété pour les L valeurs de i et les R tables, ferait perdre tout l'intérêt de la méthode. Une indexation sur $pass_{x,L}$ ramène le coût de chaque test à $O(1)$ (ou $O(\log N)$), l'attaque étant alors dominée par le recalcul des chaînes et non par les recherches.

Question 3

Non, pas forcément. Quand on tombe sur $pass_{x,L}$ après i étapes, ça veut dire que la suite de calculs partie de *inputhash* a rejoint la chaîne x à un moment donné. Mais rien ne dit qu'elle l'a rejointe dès le début.

La fonction r envoie un espace énorme (les hashes) vers un espace plus petit (les mots de passe), donc il y a forcément des collisions : deux hashes différents peuvent donner le même mot de passe. Si $inpuhash$ et $hash_{x,j}$ ont la même image par r , on rejoint la chaîne x à partir de là et on arrive sur $pass_{x,L}$, alors que $inpuhash$ n'a jamais été dans la chaîne.

chaîne x : $pass_0 \rightarrow \dots \rightarrow hash_j \xrightarrow{\quad} pass_{\{j+1\}} \rightarrow \dots \rightarrow pass_L$
 $inpuhash \dashrightarrow$ (r donne le meme mot de passe)

Dans ce cas le candidat qu'on reconstruit à partir de $pass_{x,0}$ est $pass_{x,L-i-1}$, dont le hash est $hash_{x,L-i-1} \neq inpuhash$. On appelle ça une fausse alarme. Il faut donc toujours vérifier le candidat en calculant $h(candidat)$ et en comparant avec $inpuhash$.

Question 4

Non, à condition de faire la recherche complètement, c'est-à-dire tester tous les i de 0 à $L-1$ et vérifier tous les candidats trouvés (pas seulement le premier).

Si le mot de passe cherché est $pass_{x,j}$ dans une chaîne x (avec $j \leq L-1$), alors $inpuhash = hash_{x,j}$. En appliquant r on retombe sur $pass_{x,j+1}$, puis en continuant on suit exactement la chaîne x et on arrive sur $pass_{x,L}$ au bout de $i = L-1-j$ étapes. Comme tout est déterministe, on le trouve forcément.

Donc si aucun candidat ne marche, c'est que le mot de passe n'est dans aucune chaîne. Petite exception : les $pass_{x,L}$ eux-mêmes, dont le hash n'est jamais calculé, ne sont pas couverts par la table.

Question 5

Si $pass_{x,i} = pass_{y,j}$ avec $x \neq y$ (collision), alors comme h et r sont déterministes, tout ce qui suit est identique dans les deux chaînes : $pass_{x,i+k} = pass_{y,j+k}$ pour tout k . Les deux chaînes fusionnent.

Conséquences :

- **Place et calcul gaspillés.** Tous les mots de passe après la collision sont présents deux fois. La table couvre donc moins de $N \times L$ mots de passe distincts, alors qu'on a payé le calcul de $N \times L$ hashes.
- **Si $i = j$,** les deux chaînes ont le même point d'arrivée $pass_{x,L} = pass_{y,L}$. On peut le détecter au moment du pré-calcul et jeter une des deux chaînes, mais le calcul a déjà été fait pour rien.
- **Si $i \neq j$,** les points d'arrivée sont différents, donc on ne voit pas la fusion dans la table. Le doublon est invisible et impossible à nettoyer.
- **Plus de fausses alarmes.** Lors d'une attaque, si on tombe sur $pass_{y,L}$, on reconstruit la chaîne y depuis $pass_{y,0}$, mais le mot de passe peut en fait être dans la partie de la chaîne x située avant la fusion. Le candidat est faux et on a perdu du temps de calcul.

Plus N et L sont grands, plus les collisions sont probables (paradoxe des anniversaires), donc plus on veut couvrir de mots de passe, moins la méthode est efficace. C'est le principal défaut des hashes chaînés, et c'est ce que les rainbow tables corrigent en utilisant une fonction de réduction différente à chaque colonne : deux chaînes ne fusionnent alors que si la collision arrive à la même colonne.

Question 6

Cas $pass_{x,i} = pass_{y,j}$ avec $i \neq j$. Ce n'est plus vraiment un problème. Les deux chaînes ont un mot de passe en commun, donc le hash suivant est le même (h est déterministe), mais ensuite on applique r_i dans la chaîne x et r_j dans la chaîne y . Comme $i \neq j$ ce sont deux fonctions différentes, donc en général $r_i(hash) \neq r_j(hash)$ et les chaînes se séparent tout de suite. Elles ne fusionnent pas, on a juste un seul mot de passe en double au lieu de toute la fin de chaîne.

Cas $pass_{x,i} = pass_{y,i}$ (même colonne). Là, à partir de la colonne i les deux chaînes appliquent exactement les mêmes fonctions $h, r_i, h, r_{i+1}, \dots$ sur la même valeur, donc elles sont identiques jusqu'au bout : $pass_{x,L} = pass_{y,L}$. C'est une vraie fusion, comme avec les hashes chaînés.

C'est problématique parce que la fin des deux chaînes est stockée deux fois pour rien. Mais c'est beaucoup moins grave qu'avant pour deux raisons :

- ça n'arrive que si la collision tombe exactement dans la même colonne, ce qui est L fois moins probable qu'une collision quelconque ;
- on le voit directement : les deux chaînes ont le même $pass_L$. On peut donc détecter la fusion au moment de la création de la table et rejeter une des deux chaînes, ce qui était impossible avec les hashes chaînés quand $i \neq j$.

Question 7

On suppose que h et les r_i donnent des sorties uniformes dans l'espace des 26^M mots de passe.

D'après la question 6, on a $pass_{x,L} = pass_{y,L}$ si et seulement si les deux chaînes se rencontrent dans une même colonne à un moment donné. À une colonne i fixée, la probabilité que $pass_{x,i} = pass_{y,i}$ est d'environ $\frac{1}{26^M}$. Comme il y a L colonnes possibles, la probabilité de collision entre les mots de passe finaux est d'environ

$$p \approx \frac{L}{26^M}.$$

Maintenant si on met N chaînes dans la table, il y a environ $\frac{N^2}{2}$ paires de chaînes, donc le nombre de collisions attendu est d'environ

$$\frac{N^2}{2} \cdot \frac{L}{26^M}.$$

Ça devient gênant quand ce nombre approche 1 (c'est le paradoxe des anniversaires), c'est-à-dire quand $N^2 L \approx 26^M$. L'ordre de grandeur du nombre de chaînes qu'on peut insérer facilement est donc

$$N \approx \sqrt{\frac{26^M}{L}}.$$

Avec $M = 6$ et $L = 1000$: $26^6 \approx 3,1 \cdot 10^8$, donc $N \approx \sqrt{3,1 \cdot 10^5} \approx 550$ chaînes. Au-delà, les collisions se multiplient et le programme met de plus en plus de temps à trouver des $pass_{x,0}$ qui donnent un $pass_{x,L}$ nouveau. C'est pour ça qu'on répartit les chaînes sur R tables différentes.

Question 8

Avec $M = 6$, $R = 10$, $N = 100000$, $L = 1000$, on trouve un antécédent pour **9630 hashes sur 10000** (environ 96 %).

Cela correspond-il aux attentes ? Oui. Une table couvre au plus $N \times L = 10^8$ mots de passe, et avec $R = 10$ tables la couverture brute vaut $R \cdot N \cdot L = 10^9$, pour un espace de seulement $26^6 \approx 3,1 \cdot 10^8$ mots de passe. Le rapport dépasse 1, mais à cause des fusions de chaînes on ne couvre pas tout : obtenir de l'ordre de 90–96 % est donc cohérent. Les tables se complètent, chacune rattrapant une partie des mots de passe manqués par les autres.

Temps mesurés (sur un poste ensimag).

- Création des 10 tables : environ 33 s (c'est $R \cdot N \cdot L = 10^9$ hachages).
- Attaque : environ 438 s pour les 10000 hashes, soit environ 44 ms par hash.

L'attaque d'un hash coûte de l'ordre de $R \cdot L^2/2$ hachages (pour chaque table, on repart de chaque colonne). Un hash trouvé tôt est plus rapide (on s'arrête dès qu'on a vérifié le bon candidat) ; un hash non trouvé fait toute la recherche.

Question 10

La technique standard est le **sel** (*salt*). À la création d'un compte, on tire une valeur aléatoire unique (le sel), et on stocke $h(\text{sel} \parallel \text{mot de passe})$ avec le sel en clair à côté. Deux utilisateurs ayant le même mot de passe obtiennent des hashes différents.

Cela casse précisément l'attaque par base pré-calculée (rainbow tables incluses) : la table devrait être recalculée *pour chaque valeur de sel*, donc le pré-calcul partagé entre systèmes ou entre comptes ne sert plus à rien. Il faut un sel suffisamment long et vraiment aléatoire pour que deux comptes n'aient jamais le même.

On combine souvent le sel avec une **fonction de hachage lente et paramétrable** (bcrypt, scrypt, Argon2, PBKDF2), qui rend chaque calcul de hash coûteux : cela ne vise plus la base pré-calculée mais ralentit aussi la force brute.

On peut aussi ajouter un **poivre** (*pepper*), un secret global stocké séparément de la base.